Qfóton | Master Video Recording Guide

Exact 2-Minute Step-by-Step Teleprompter & On-Screen Actions for QuantumHacks

---

■■ STEP 0: SETUP CHECKLIST (30 Seconds Before Recording)

1. Open Terminal:	Navigate to `C:\Users\25beevdt047\.gemini\antigravity\scratch\PhotoQ`
2. Pre-Type Command:	Type **`python simulate_chip.py`** in terminal (**DO NOT press Enter yet**).
3. Open Browser:	Have your GitHub repository tab open: `https://github.com/atharveeee-netizen/qfoton`
4. Start Recorder:	Press **Win + Alt + R** (Windows Game Bar) or open Loom / OBS.

■■ PHASE 1 [0:00 – 0:35]: Interactive Studio & Simulink DACs

HAND ACTIONS:

1. Hit Record.
2. Read Opening Script.
3. Press **Enter** on **`python simulate_chip.py`**.
4. Click **[■ Fire Pulse]** button on GUI.
5. Click **[■ Simulink Model]** button (DAC plot pops up).
6. Click the **X** at top right to close windows.

WHAT TO SPEAK:

"Hi everyone. This is Qfóton.

Superconducting quantum processors require dilution refrigerators cooled to 15 millikelvin. Photons do not suffer from ambient heat, so our silicon photonic processors operate at room temperature (300 K) at the speed of light along optical waveguides.

Here in Qfóton Studio, we simulate a 4-qubit Quantum Teleportation protocol. Clicking Fire Pulse animates single-photon wavepackets through directional couplers and thermo-optic phase shifters with 99.6% fidelity. Clicking Simulink Model displays our physical electro-thermal DAC driving voltages and power dissipation profile."

■■ PHASE 2 [0:35 – 1:05]: Carolan et al. Science (2015) Reproduction

HAND ACTIONS:

1. Type in terminal: **`python simulate_science_2015_chip.py`** and press **Enter**.
2. 4-Panel scientific dashboard pops up on screen.
3. Read Script while showing Panel B fidelity chart.
4. Click the **X** at top right to close window.

WHAT TO SPEAK:

"Next, we demonstrate exact experimental validation against the landmark paper: Carolan et al., Science 2015.

Instead of ideal math, we inject real cleanroom parameters: 0.148 dB/cm waveguide loss, 3-nanometer sidewall roughness, and 89.2% SNSPD detector efficiency. As shown in Panel B, Qfóton's simulated fidelity of 99.40% matches the published Bristol laboratory experiment within 0.05%."

■■ PHASE 3 [1:05 – 1:30]: Universal Custom Chip Gateway & 3D Tomography

HAND ACTIONS:	1. Type in terminal: **`python simulate_custom_chip.py --preset ghz3`** and press **Enter**. 2. 3D Quantum State Tomography (Re[ρ]) bar graph pops up. 3. Read Script while showing 3D density matrix. 4. Click the **X** at top right to close window.
WHAT TO SPEAK:	*"Qfóton also includes a universal custom chip gateway. Any user can paste an OpenQASM circuit or choose presets like this 3-qubit GHZ state.* *Our compiler factors the algorithm into physical silicon MZIs, runs our inverse-coupling optimizer to cancel out 18% thermal heat bleeding, and plots the reconstructed 3D Quantum State Tomography density matrix."*

■■ PHASE 4 [1:30 – 1:50]: Full 12-Stage Physics Engine & Conclusion

HAND ACTIONS:	1. Type in terminal: **`python run_demo.py`** and press **Enter**. 2. Scroll down terminal output showing ASCII physics tables. 3. Switch window (**Alt + Tab**) to show your **GitHub repository** in browser. 4. Hit **Stop Recording**!
WHAT TO SPEAK:	*"Finally, run_demo.py executes our complete 12-stage physical simulation suite: on-chip SFWM photon sources, Nature 2024 topological protection surviving 25% physical damage, Boson Sampling permanent speedup, and true quantum random number generation with NIST compliance.* *Qfóton is open-source and live on GitHub at github.com/atharveeee-netizen/qfoton. Thank you for watching."*

■ DEVPOST SUBMISSION CHEAT SHEET (Copy & Paste Ready)

Project Title:	Qfóton: Linear Optical Quantum Computing & Silicon Photonic Chip Simulator
Elevator Pitch:	A high-performance Python library for compiling linear optical quantum circuits, topologically protected waveguides, and room-temperature silicon photonic chips.
GitHub URL:	`https://github.com/atharveeee-netizen/qfoton`
Built With:	python, quantum-computing, linear-optics, numpy, scipy, boson-sampling, hardware-simulation